ESSENTIAL TRAINING SERIES

K3s

Lightweight Kubernetes in Production

Install · Deploy · Network · Storage · Production

8 Exercises | 4 Hours | Intermediate

COURSE CONTENTS

Hands-on Lab — Infrastructure & Container Orchestration

CNCF Certified Kubernetes Distribution

Introduction to K3s

K3s is a CNCF-certified Kubernetes distribution designed for resource-constrained environments (edge, IoT, Raspberry Pi) and rapid development. It packages all Kubernetes components into a single binary of approximately 40 MB, while remaining 100% compatible with the standard Kubernetes API.

Why K3s?

- Single binary — no external dependencies to manage
- Installs in under 60 seconds on any Linux machine
- SQLite as the default database (etcd optional for HA)
- Native ARM architecture support (Raspberry Pi, etc.)
- Traefik, Flannel CNI and CoreDNS included and pre-configured

K3s vs Standard Kubernetes

Feature	K3s	Kubernetes (kubeadm)
Binary size	~40 MB	~100+ MB
Database	SQLite (default)	etcd (required)
Install time	< 1 minute	5-15 minutes
Minimum RAM	~512 MB	~2 GB
ARM native	Yes	Manual configuration
Built-in Ingress	Traefik v3	None (must install)

Built-in Components

Component	Role	Port(s)
API Server	Kubernetes REST entry point	6443/tcp
Scheduler	Pod placement on nodes	—
Controller Manager	Control loops (replica, node...)	—
SQLite / etcd	Cluster state store	2379/tcp (etcd)
Containerd	Container runtime	—

Flannel CNI	Pod overlay network	8472/udp
Traefik v3	Ingress controller	80, 443/tcp
CoreDNS	Internal cluster DNS	53/udp+tcp

EXERCISE 1

Installing K3s

30 min duration

Easy level

Context

You have a Linux machine (Ubuntu 22.04 or Debian 12). The goal is to install K3s in server mode (control plane + worker on the same node) and verify the cluster is up and running.

1.1 · Check prerequisites

```
# Available memory (min 512 MB, recommended 1 GB)
free -h
# Disk space (min 1 GB in /var/lib/rancher)
df -h /
# Kernel version (min 4.x)
uname -r
```

1.2 · Install the server node

1. Run the official install script:

```
# Download and run the K3s install script -- sets up server + agent on one node
curl -sfL https://get.k3s.io | sh -
```

2. Check the service is active and the node is ready:

```
# Confirm the k3s systemd service is running
systemctl status k3s
# List nodes -- should show one node in Ready state
sudo k3s kubectl get nodes
```

1.3 · Configure kubectl

To use kubectl directly without the k3s prefix, export the kubeconfig:

```
# Point kubectl to the K3s-generated kubeconfig file
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
# Persist the setting so it survives new terminal sessions
echo 'export KUBECONFIG=/etc/rancher/k3s/k3s.yaml' >> ~/.bashrc
# Reload shell config to apply the change immediately
source ~/.bashrc

# Verify kubectl can reach the cluster and list nodes
kubectl get nodes -o wide
# Show the API server and CoreDNS addresses
kubectl cluster-info
```

> ☆ **INFO**
> The kubeconfig is auto-generated at /etc/rancher/k3s/k3s.yaml. It contains TLS certificates and the API server URL. To use it from a remote machine: scp user@server:/etc/rancher/k3s/k3s.yaml ~/.kube/config

> ✝ **QUESTIONS**
> 1. What is the node status after installation? What role is assigned?
> 2. Which processes are started by K3s? (hint: ps aux | grep k3s)
> 3. Which container runtime is used by default? How can you verify it?
> 4. Where is the cluster database stored by default?

EXERCISE 2 | Multi-Node Cluster | 45 min duration | Intermediate level

Context

You want to build a multi-node K3s cluster. Depending on your environment, choose the scenario that applies: Scenario A for a single machine with Docker (WSL2/Windows or Linux desktop), Scenario B for two real Linux VMs or bare-metal servers.

2.0 · Prerequisites

SCENARIO A · Single machine (k3d / WSL2)

```
# Install k3d (K3s in Docker)
curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
k3d version

# Create a cluster: 1 server + 2 agents (all as Docker containers)
k3d cluster create mycluster --servers 1 --agents 2

# kubeconfig is auto-configured -- no token needed
kubectl get nodes -o wide
```

> ☆ **INFO**
> k3d runs K3s nodes as Docker containers on a single machine. Each gets its own IP on a Docker bridge network. Ideal for WSL2, laptops, and CI environments. No manual token or K3S_URL required.

SCENARIO B · Two real VMs or bare-metal servers

```
# On the SERVER node -- retrieve the join token
sudo cat /var/lib/rancher/k3s/server/node-token

# On the AGENT node -- join the cluster (replace SERVER_IP and TOKEN)
curl -sfL https://get.k3s.io | \
  K3S_URL=https://<SERVER_IP>:6443 \
  K3S_TOKEN=<TOKEN> sh -

# From the server -- verify both nodes appear
kubectl get nodes -o wide
```

> ☆ **WARNING**
> On WSL2, K3S_URL=https://127.0.0.1:6443 fails · WSL2 cannot route loopback between VMs. Use Scenario A (k3d) instead. Scenario B requires two machines with real routable IPs and open ports: 6443/tcp, 8472/udp, 10250/tcp.

2.1 · Verify cluster nodes (both scenarios)

```
# Expected output for Scenario A (k3d):
# k3d-mycluster-server-0   Ready   control-plane,master
# k3d-mycluster-agent-0    Ready   <none>
# k3d-mycluster-agent-1    Ready   <none>

# Expected output for Scenario B (real VMs):
# my-server   Ready   control-plane,master
# my-agent    Ready   <none>

kubectl get nodes -o wide
```

2.2 · Verify inter-pod connectivity

```
# Deploy a test pod on the first node -- keeps running for 1 hour
kubectl run ping-test --image=busybox --restart=Never -- sleep 3600
# Check which node it landed on and what IP it got
kubectl get pod ping-test -o wide

# Scenario A -- force second pod onto agent node (nodeName)
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: ping-test2
spec:
  nodeName: k3d-mycluster-agent-0
  containers:
  - name: busybox
    image: busybox
    command: ["sleep", "3600"]
EOF
```

```
# Scenario B -- nodeSelector: kubernetes.io/hostname: <agent-hostname>
kubectl get pod ping-test2 -o wide

# Cross-node ping (replace with actual pod IP)
kubectl exec -it ping-test -- ping <PING_TEST2_IP>
# Expected: 64 bytes reply -- Flannel VXLAN cross-node confirmed
```

QUESTIONS

1. What is the key difference between k3d and a real multi-VM K3s cluster?
2. Which ports must be open between nodes in Scenario B and why?
3. How do you force a pod onto a specific node: nodeName vs nodeSelector vs affinity?
4. What Docker network does k3d use? How can you inspect it with docker network inspect?

EXERCISE 3 — First Deployment — 30 min duration — Easy level

3.1 · Deploy NGINX

Create the file nginx-deploy.yaml and apply it:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-demo
  template:
    metadata:
      labels:
        app: nginx-demo
    spec:
      containers:
      - name: nginx
        image: nginx:1.27-alpine
        ports:
        - containerPort: 80
        resources:
          requests: { memory: '64Mi', cpu: '50m' }
          limits:   { memory: '128Mi', cpu: '100m' }
```

```
# Apply the manifest -- creates the Deployment and its pods
kubectl apply -f nginx-deploy.yaml
# List pods with their assigned node and IP
kubectl get pods -l app=nginx-demo -o wide
# Wait and confirm all replicas finish rolling out successfully
kubectl rollout status deploy/nginx-demo
```

3.2 · Expose and test

```
# Create a ClusterIP service -- makes the deployment reachable by name inside the cluster
kubectl expose deploy nginx-demo --port=80 --name=nginx-svc

# Spin up a temporary pod to test -- ClusterIP is not reachable from outside the cluster
kubectl run curl-test --image=curlimages/curl --restart=Never --rm -it \
  -- curl http://nginx-svc
```

3.3 · Update & rollback

```
# Update to a DIFFERENT version -- this creates a new rollout revision
kubectl set image deploy/nginx-demo nginx=nginx:1.25-alpine
# Watch until the new pods are fully up and old ones are terminated
kubectl rollout status deploy/nginx-demo

# View the full history of rollouts for this deployment
kubectl rollout history deploy/nginx-demo
# Roll back to the previous version if something went wrong
kubectl rollout undo deploy/nginx-demo
```

QUESTIONS

1. How many ReplicaSets are created after an update? Why?
2. What is the difference between ClusterIP, NodePort and LoadBalancer?
3. How do you scale the deployment to 5 replicas in a single command?

EXERCISE 4

Ingress with Traefik

30 min duration

Intermediate level

Context

K3s includes Traefik v3 as the default Ingress controller on ports 80 and 443. You will expose the NGINX application via an Ingress rule with hostname-based routing.

4.1 · Create the Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-ingress
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: web
spec:
  rules:
  - host: nginx.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: nginx-svc
            port:
              number: 80
```

```
# Apply the Ingress rule to Traefik
kubectl apply -f nginx-ingress.yaml
# Confirm the Ingress was registered -- note the ADDRESS column
kubectl get ingress

# Get your node IP from the ADDRESS column above (pick the first one)
# For k3d it will be something like 172.19.0.2
# Replace <NODE_IP> with that address, e.g: echo '172.19.0.2 nginx.local'
echo '<NODE_IP> nginx.local' | sudo tee -a /etc/hosts
# Test the Ingress route -- should return the nginx welcome page HTML
curl http://nginx.local
```

4.2 · Traefik Dashboard

```
# On bare-metal K3s: port-forward works directly
# kubectl port-forward -n kube-system svc/traefik 9000:9000
# Then open: http://localhost:9000/dashboard/

# On k3d / WSL2: use a Traefik IngressRoute CRD (port-forward fails with 502)
# Step 1 -- save to file using single-quoted EOF to preserve backticks
cat > traefik-dashboard.yaml << 'EOF'
apiVersion: traefik.io/v1alpha1
kind: IngressRoute
metadata:
  name: traefik-dashboard
  namespace: kube-system
spec:
  entryPoints:
    - web
  routes:
    - match: Host(\`traefik.local\`) && (PathPrefix(\`/dashboard\`) || PathPrefix(\`/api\`))
      kind: Rule
      services:
        - name: api@internal
          kind: TraefikService
EOF
# Step 2 -- apply the IngressRoute
kubectl apply -f traefik-dashboard.yaml
# Step 3 -- get the ADDRESS IP
kubectl get ingress -A
# Step 4 -- add to WSL2 hosts (replace <ADDRESS_IP>)
echo '<ADDRESS_IP> traefik.local' | sudo tee -a /etc/hosts
# Step 4b -- test from WSL2 terminal (should return Traefik dashboard HTML)
curl http://traefik.local/dashboard/

# Step 5 -- add to Windows hosts (PowerShell as Administrator)
# Add-Content C:\\Windows\\System32\\drivers\\etc\\hosts '<ADDRESS_IP> traefik.local'
# Step 6 -- open in browser: http://traefik.local/dashboard/
```

② TIP · What you should see in the Traefik dashboard

1. Dashboard: 4 entrypoints (METRICS :9100, TRAEFIK :8080, WEB :8000, WEBSECURE :8443)
2. HTTP Routers: 5 routers all green, 0 warnings, 0 errors
3. Router nginx.local -> default-nginx-svc-80 (your 2 nginx replicas) on web entrypoint
4. Router traefik.local -> api@internal (the dashboard itself) on web entrypoint
5. HTTP Services: 7 services, default-nginx-svc-80 shows 2 servers (your nginx pods)
6. On k3d/WSL2: IP changes on every restart -- always run kubectl get ingress -A
and update both /etc/hosts (WSL2) and Windows hosts file with the new IP

QUESTIONS

1. What is the difference between an Ingress and a LoadBalancer service?
2. How do you enable HTTPS with a self-signed certificate via Traefik?
3. What happens if two Ingress resources define the same host?

EXERCISE 5

Persistent Storage

30 min duration

Intermediate level

Context

K3s includes the local-path-provisioner which automatically creates volumes on the local node. You will deploy PostgreSQL with persistent storage and verify that data survives a pod restart.

5.1 · PersistentVolumeClaim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: local-path
  resources:
    requests:
      storage: 1Gi
```

```
# Save the above to postgres-pvc.yaml and apply it
kubectl apply -f postgres-pvc.yaml
# Verify the PVC was created and is bound
kubectl get pvc
```

5.2 · Deploy PostgreSQL

```
# Excerpt from the deployment manifest
containers:
- name: postgres
  image: postgres:17-alpine
  env:
  - name: POSTGRES_PASSWORD
    value: 'mypassword'
  volumeMounts:
  - mountPath: /var/lib/postgresql/data
    name: postgres-data
volumes:
- name: postgres-data
  persistentVolumeClaim:
    claimName: postgres-pvc
```

```
# Save the above to postgres-deploy.yaml and apply it
kubectl apply -f postgres-deploy.yaml
# Verify the pod is running
kubectl get pods
```

```
# Open a psql shell inside the running postgres pod
kubectl exec -it <POSTGRES_POD> -- psql -U postgres
CREATE DATABASE testdb;
\c testdb
CREATE TABLE users (id SERIAL, name TEXT);
INSERT INTO users (name) VALUES ('Alice');
\q

# Delete the pod -- the Deployment will automatically recreate it
kubectl delete pod <POSTGRES_POD>

# Reconnect to the new pod and verify the data is still there
kubectl exec -it <NEW_POD> -- psql -U postgres -d testdb -c 'SELECT * FROM users;';
```

☆ **WARNING**

The local-path provisioner stores data on the local node only. If that node fails, the data is lost. For high availability, use Longhorn or a shared NFS server.

✝ **QUESTIONS**

1. Where is data physically stored on the host node?
2. What happens if the node hosting the PV goes down?
3. What solution should you use for shared storage across multiple nodes?

EXERCISE 6

ConfigMaps & Secrets

20 min duration

Easy level

6.1 · ConfigMap

```
# Create a ConfigMap with non-sensitive app configuration key-value pairs
kubectl create configmap app-config \
  --from-literal=DB_HOST=postgres \
  --from-literal=DB_PORT=5432 \
  --from-literal=APP_ENV=production

# Inspect the ConfigMap to verify all keys were stored
kubectl describe configmap app-config
```

6.2 · Secret

```
# Create a Secret -- values are base64-encoded at rest in etcd
kubectl create secret generic app-secret \
  --from-literal=DB_PASSWORD=my-secret-password \
  --from-literal=API_KEY=api-key-12345

# Decode a base64-encoded value to verify it was stored correctly
kubectl get secret app-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
```

6.3 · Inject into a pod

```
# Create a test pod injecting both ConfigMap and Secret as env vars
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: app-env-test
spec:
  containers:
  - name: app
    image: busybox
    command: ["sleep","3600"]
    envFrom:
    - configMapRef:
        name: app-config
    - secretRef:
        name: app-secret
EOF

# Verify all 5 variables were injected (from ConfigMap and Secret)
kubectl exec app-env-test -- env | grep -E 'DB_HOST|DB_PORT|APP_ENV|DB_PASSWORD|API_KEY'

# Clean up
kubectl delete pod app-env-test
```

☆ **WARNING**

Never store secrets in plain text in YAML manifests committed to Git. In production, prefer Sealed Secrets, External Secrets Operator or HashiCorp Vault.

EXERCISE 7

Monitoring & Debugging

30 min duration

Intermediate level

7.1 · Diagnostic commands

```
# List every resource in every namespace -- quick full cluster overview
kubectl get all -A

# Show recent events sorted by time -- useful to spot failures or warnings
kubectl get events -A --sort-by=.lastTimestamp | tail -20

# Show CPU and RAM usage per node
kubectl top nodes
# Show CPU and RAM usage per pod across all namespaces
kubectl top pods -A
```

7.2 · Analyze a failing pod

```
# Create a pod with a bad image -- it will fail to start
kubectl run bad-pod --image=nginx:doesnotexist

# Watch the pod status change in real time (Ctrl+C to stop)
kubectl get pod bad-pod -w

# Read the full event log to understand why the pod failed
kubectl describe pod bad-pod

# Read the logs from the previous crashed container instance
kubectl logs bad-pod --previous
# Clean up the failed pod
kubectl delete pod bad-pod
```

7.3 · Network & DNS debugging

```
# Test internal DNS -- confirms CoreDNS is resolving service names
# Use the full DNS name: <service>.<namespace>.svc.cluster.local
kubectl run dns-test --image=busybox --restart=Never --rm -it \
  -- nslookup kubernetes.default.svc.cluster.local
# Expected: Name: kubernetes.default.svc.cluster.local / Address: <CLUSTER-IP>

# Test connectivity to a service using its full DNS name
kubectl run curl-debug --image=curlimages/curl --restart=Never --rm -it \
  -- curl -v http://nginx-svc.default.svc.cluster.local

# Stream live K3s server logs -- useful for node-level issues
journalctl -u k3s -f --since '5 minutes ago'
```

QUESTIONS

1. What are the different possible states of a pod (phase)?
2. What is the difference between CrashLoopBackOff and ImagePullBackOff?
3. How do you stream logs from multiple pods sharing the same label?

EXERCISE 8 — High Availability & Production — 45 min duration — Advanced level

8.1 · HA cluster with etcd

For a production cluster, replace SQLite with embedded etcd. Minimum 3 server nodes required to guarantee quorum. Choose your scenario:

SCENARIO A · k3d / WSL2 (simulate HA locally)

```
# Delete current cluster and recreate with 3 server nodes
# k3d automatically enables embedded etcd with --servers 3
k3d cluster delete mycluster
k3d cluster create mycluster --servers 3 --agents 2

# Verify -- all 3 server nodes should show control-plane,master
kubectl get nodes -o wide
```

INFO

k3d with --servers 3 automatically bootstraps embedded etcd and joins all server nodes. No manual --cluster-init or token needed. This is the k3d equivalent of a 3-node HA production cluster.

SCENARIO B · Bare-metal / real VMs (production)

```
# Server 1 -- initialize etcd cluster
curl -sfL https://get.k3s.io | sh -s - server \
  --cluster-init \
  --tls-san <LOAD_BALANCER_IP>

# Servers 2 and 3 -- join the etcd cluster
curl -sfL https://get.k3s.io | sh -s - server \
  --server https://<SERVER_1_IP>:6443 \
  --token <TOKEN>
```

8.2 · ResourceQuota per namespace

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: ns-quota
  namespace: production
spec:
  hard:
    pods: '20'
    requests.cpu: '4'
    requests.memory: 8Gi
    limits.cpu: '8'
    limits.memory: 16Gi
```

```
# Create the production namespace then apply the quota
kubectl create namespace production
kubectl apply -f resource-quota.yaml

# Simulate used values -- deploy a pod with resource requests into production
kubectl run test-quota --image=nginx --namespace=production \
  --requests='cpu=100m,memory=128Mi' --limits='cpu=200m,memory=256Mi'

# Verify -- Used column should now show non-zero values
kubectl describe resourcequota ns-quota -n production
```

8.3 · etcd backup & restore

```
# Backup -- creates a snapshot of the etcd database
k3s etcd-snapshot save --name backup-$(date +%Y%m%d)
# List existing snapshots to confirm the backup was created
ls /var/lib/rancher/k3s/server/db/snapshots/

# Restore -- resets the cluster state from a snapshot file
k3s server --cluster-reset \
  --cluster-reset-restore-path=<SNAPSHOT_PATH>
```

8.4 · Upgrading K3s

```
# Check the currently installed K3s version
k3s --version

# Upgrade to a specific version (always pin the version!)
curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=v1.35.3+k3s1 sh -
```

❷ TIP

Always set INSTALL_K3S_VERSION in production. Test on staging before applying to production. Upgrade agents first, then the server.

QUESTIONS

1. How many server nodes are required minimum for an etcd quorum? Why?
2. What is the difference between Guaranteed, Burstable and BestEffort QoS?
3. Which tools would you recommend for monitoring a K3s cluster?

/etc/rancher/k3s/k3s.yaml	Kubeconfig — cluster access
/var/lib/rancher/k3s/server/node-token	Agent join token
/var/lib/rancher/k3s/server/db/	SQLite or etcd database
/var/lib/rancher/k3s/server/db/snapshots/	Automatic etcd snapshots
/usr/local/bin/k3s-uninstall.sh	Uninstall script (server)
/usr/local/bin/k3s-agent-uninstall.sh	Uninstall script (agent)

INFO

Official docs: https://docs.k3s.io | GitHub: https://github.com/k3s-io/k3s | CNCF: https://www.cncf.io/projects/k3s/

WHAT YOU WILL LEARN

- Install K3s on a VM or bare-metal server in under 2 minutes
- Build a multi-node cluster with server and agent nodes
- Deploy, scale and update containerized applications
- Configure Traefik as an Ingress controller for HTTP/HTTPS
- Manage persistent storage with PVCs and local-path-provisioner
- Secure configurations with Secrets and ConfigMaps
- Diagnose network, pod and cluster issues
- Set up high availability with embedded etcd

PREREQUISITES

- Linux (Ubuntu 22.04+)
- Docker basics
- TCP/IP & HTTP

4h
Duration

8
Exercises

Intermediate
Level

"From zero to cluster in under 60 seconds."

K3s — The power of Kubernetes, without the complexity